

CA3 Reflective Document: Advanced 3D Game Development

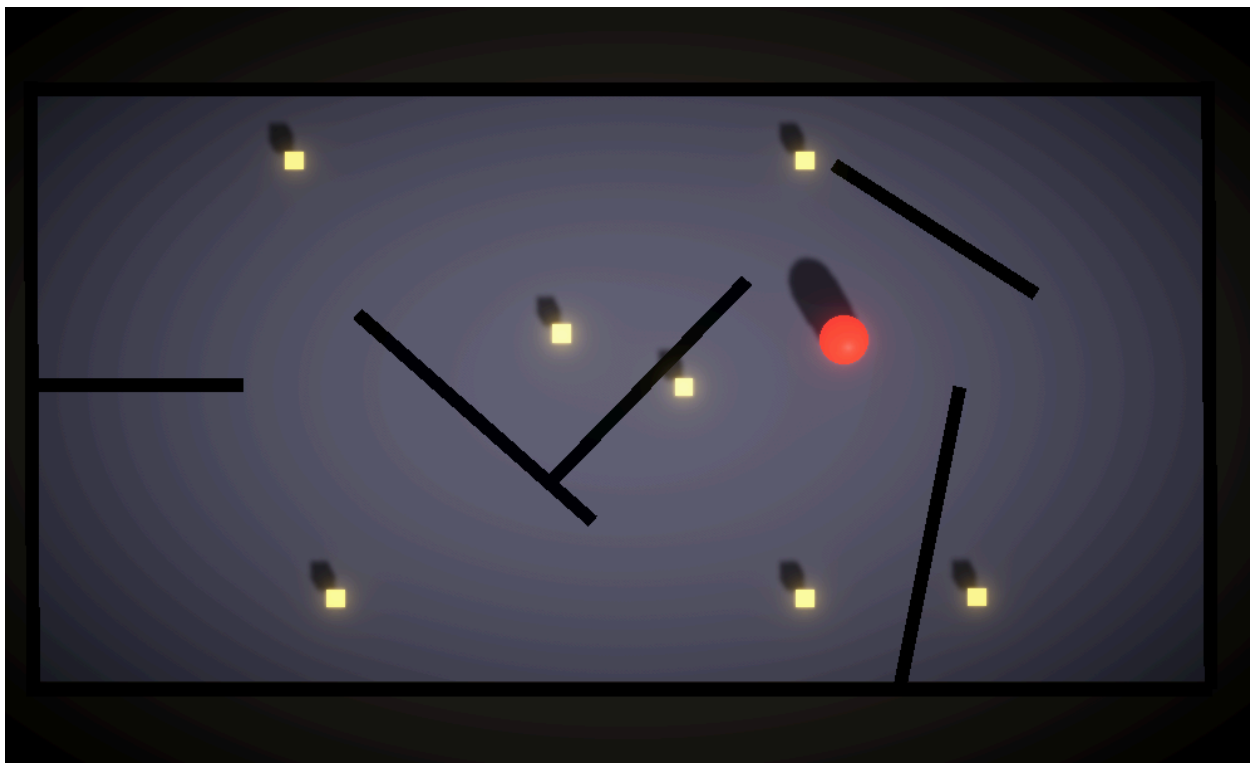
Name: Wolfgang Romanowski

Student Number: 20101931

Repository: <https://github.com/Wolfgang-Romanowski/GameDev-Unity-CA-Sem2>

Branch: feature/ca3-network

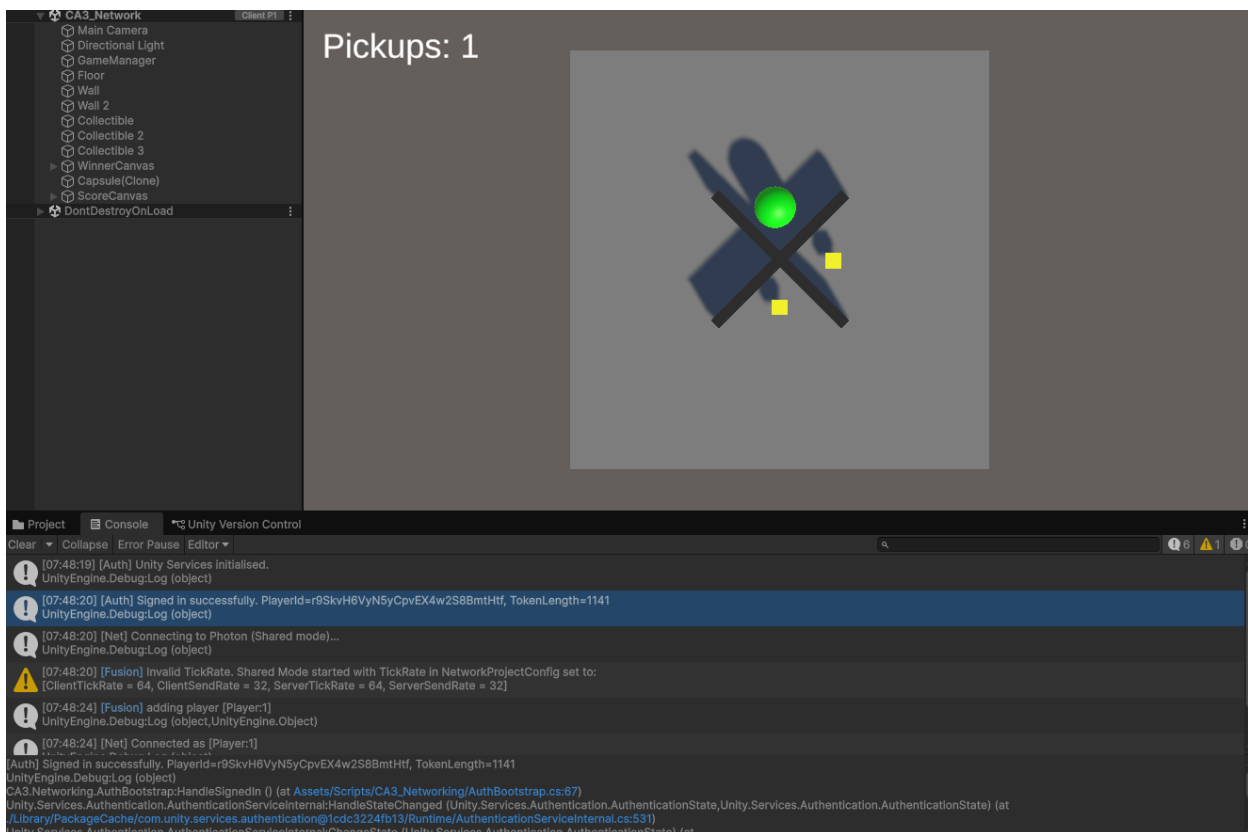
Scene: Assets/Scenes/CA3_Network.unity



Section 1: Rendering Foundations

This document is the natural evolution of the technical note submitted for CA1 and the critical discussion submitted for CA2, revised and extended to reflect the integrated state of the project at CA3 submission

CA1 and CA3_Network share the same URP project baseline: HDR enabled, SSAO renderer feature, ACES tonemapping, dark ambient, desaturated palette. Beyond that they go in different directions. CA1 is a cinematic rendering showcase built around a multi-light setup, baked GI, a baked reflection probe, and a custom dissolve ShaderGraph. CA3_Network is a networking sandbox.



Single directional light, no custom shaders, no reflection probe, orthographic top-down camera, post-processing graded for gameplay readability not mood. The aesthetic intent carries across. The visual complexity does not. That was a deliberate trade-off.

Camera and projection:

CA1 uses a perspective hero camera with a custom skybox. CA3_Network uses an orthographic camera (Size: 8, rotation X: 90) looking straight down with Post Processing enabled on the PC_Renderer.

I chose orthographic because it gives unambiguous top-down spatial readability of the guard, players, and collectibles. A perspective camera would add visual interest but make relative positions harder to read at a glance, which matters more than cinematic framing in a networking test scene.

Lighting:

CA1's lighting is a directed multi-light composition: a teal-tinted directional key (intensity 1.5) plus three coloured point lights, a blue rim, an orange window glow, and a red-orange fill, supported by a baked reflection probe and full baked GI with 1024² lightmaps. CA3_Network uses a single directional light at rotation X: 50, Y: -30, intensity 0.8 with a cool cyan tint, soft shadows at strength 0.7, and gradient ambient at full intensity. No point lights, no reflection probe, no baked lightmap beside the scene.

The geometry reads flat compared to CA1. That is honest. The scene exists to test networking correctness and the lighting reflects that. The main lighting contribution unique to CA3 is the dynamic emissive system on gameplay objects. NetworkGuardVisuals.cs interpolates the guard's `_EmissionColor` between near-black (`Color(0.02f, 0.02f, 0.02f)`) in patrol and HDR red (`Color(11.5f, 0f, 0f)`) in chase via `Color.Lerp`. Pickup collectibles emit constant warm yellow. These are the only motivated light sources in the scene. `_EMISSION` must be explicitly enabled via `material.EnableKeyword("_EMISSION")` before `SetColor("_EmissionColor", ...)` takes effect at runtime in URP. Omitting this call causes the colour change to silently do nothing. I handle this in

`NetworkGuardVisuals.Spawned()` on a per-renderer instance material to avoid shared material mutation across objects.



Materials:

CA1 applies SG_A2_DissolveBurn_Lit with authored diffuse and normal map textures imported correctly with sRGB on/off per channel. CA3_Network uses four custom URP Lit materials (Mat_Wall, Mat_Floor, Mat_Collectible, Mat_Guard) with no textures. Every slot (_BaseMap, _BumpMap, _MetallicGlossMap, _OcclusionMap) is empty. Surface properties are set by colour and numeric values only (_Metallic 0, _Smoothness 0.5, _BaseColor black). Mat_Collectible bakes the warm yellow emission directly into the asset (_EmissionColor (4, 2.48, 0.057), _EMISSION keyword enabled). Mat_Guard's emission is zero in the asset and driven at runtime by NetworkGuardVisuals. It is a meaningful reduction in surface fidelity versus CA1. The geometry exists to define collision and navigation space, not to demonstrate material authoring.

Post-processing:

Both scenes use ACES tonemapping but the grades are opposite. CA1 uses subtle warm bloom (Threshold 1.3, Intensity 0.3, warm orange tint) and a violet vignette (Intensity 0.2). That is atmospheric and moody. CA3_Network's scene-local Volume Profile is tuned for readability. Bloom is at Threshold 0.9 and Intensity 1.2: hotter, lower threshold, untinted. The lower threshold means even the collectibles' modest emission contributes to the bloom pass. Vignette is at Intensity 0.4, stronger than CA1, because orthographic projection has no natural depth falloff to focus attention. The Color Adjustments override (absent from CA1) sets Post Exposure to -0.5, Contrast to +15, and Saturation to -10. The exposure reduction pushes non-emissive surfaces toward shadow. The contrast sharpens the boundary between lit and unlit areas. The desaturation lets the coloured emissive elements, the guard's red glow, the yellow collectibles, the colour-coded player capsules from NetworkPlayerAppearance.cs, read clearly against a neutral background.

Unreal Engine update:

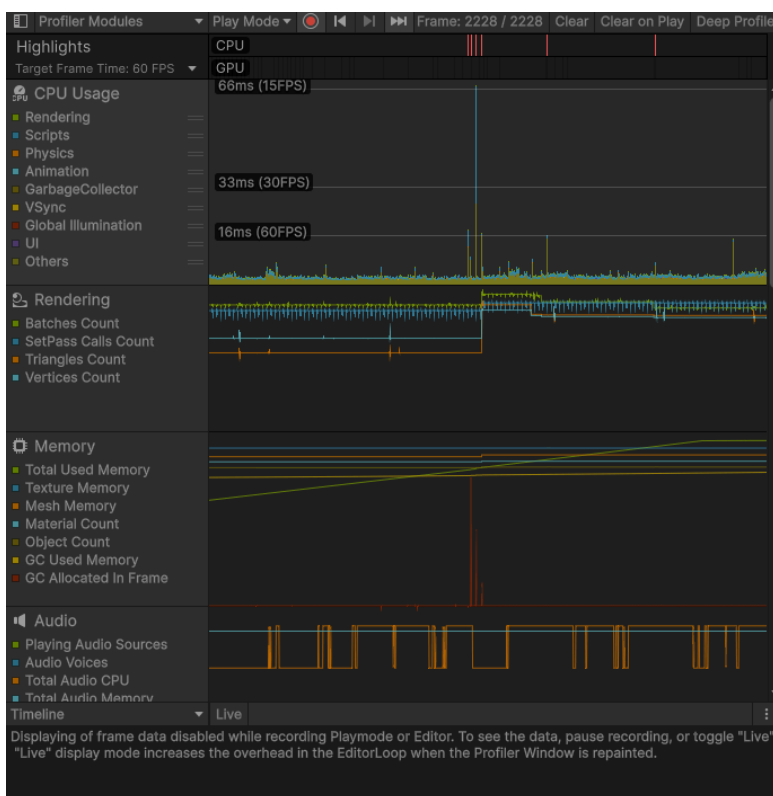
The CA1 comparison noted that Unreal's Material Editor separates Opacity Mask and Emissive more explicitly. An additional observation from CA3: Unreal handles runtime emissive updates through dynamic material instances and Material Parameter Collections without requiring keyword management. The equivalent of Unity's EnableKeyword("_EMISSION") plus SetColor("_EmissionColor", ...) is just setting a vector parameter on a dynamic material instance. That is cleaner when driving emissive state from gameplay code.

Section 2: Optimisation and Constraints

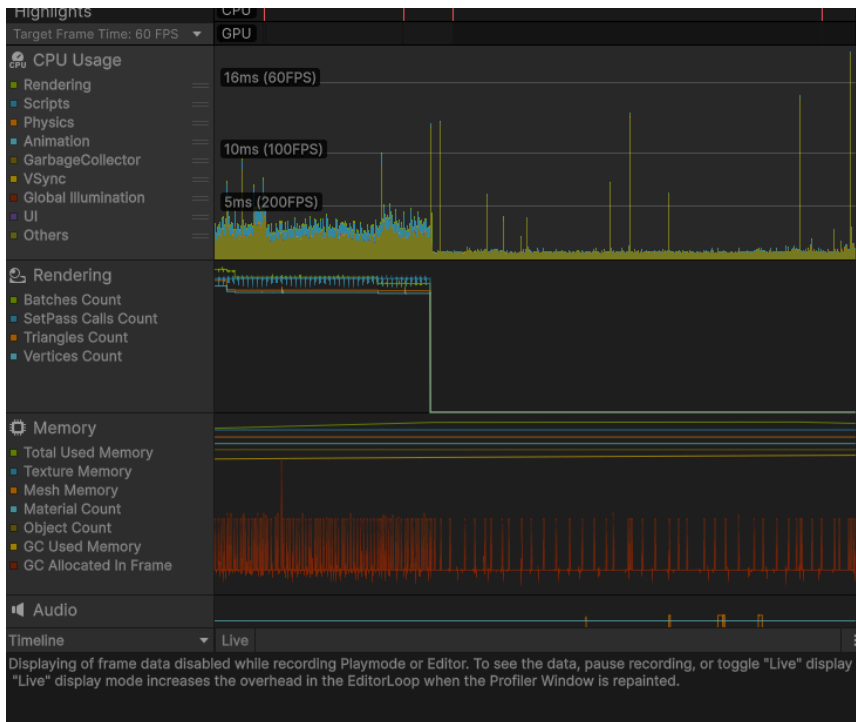
Guard perception throttle:

The initial implementation called `FindClosestPlayerInRange()` on every Fusion tick in `NetworkGuard.FixedUpdateNetwork()`. That runs at 64 Hz on the authority peer. The method iterates `Runner.ActivePlayers`, does a `Vector3.Distance` check per player, and accesses `NetworkPlayerStun`. At four players each call is cheap but it was generating allocations every tick. The GC Allocated in Frame row in the before capture (`CA3_Network_Profile_Before.png`) shows dense orange spikes throughout the session. CPU frame time was spiking to 66ms worst case. I added a `perceptionTickInterval` field defaulting to 4 and exposed as `[SerializeField]`. Perception now runs at 16 Hz. `agent.SetDestination` still runs every tick so pathfinding stays smooth toward the confirmed target. The after capture (`CA3_Network_Profile_After.png`) shows CPU frame time settling around 5ms with the allocation spikes gone. At chase speed (5 m/s) a player moves roughly 0.31 m between perception ticks at 16 Hz. That is below the detection threshold granularity so guard responsiveness is unchanged.

Before

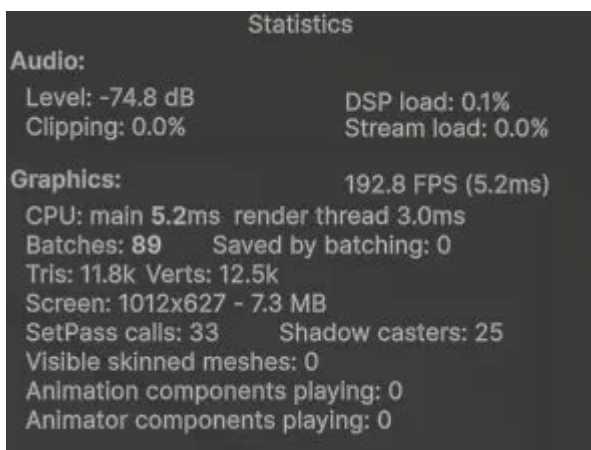


After:



NavMesh. The arena layout is fixed so I baked the NavMesh statically. Dynamic obstacles and off-mesh links were not used. NavMesh rebaking at runtime is expensive and unnecessary here. The static bake produces deterministic paths and was not a source of runtime cost. Agent parameters (patrol speed 2, chase speed 5) are set in the Inspector.

[Networked] property cost. The guard uses two [Networked] properties: CurrentWaypointIndex (int) and State (enum). Both are small value types. Replicating the NavMesh path or agent velocity directly would be much more expensive. NetworkGuardVisuals reads guard.State in Render() with no allocation. That is the correct hook for visual state that should not affect simulation.



The Stats overlay confirms 89 batches and 33 SetPass calls at runtime, with 11.8k triangles and 12.5k vertices which are modest figures for a networked arena of this scope. No LOD groups or

occlusion culling are configured because the geometry is simple enough that neither system would provide meaningful savings at this scale

A reasonable real-time performance target for a two-to-four player networked arena of this scope is 60fps sustained on mid-range hardware. The scene meets this target: CPU frame time settles around 5ms after the perception throttle optimisation, well within the 16.6ms budget for 60fps. The bottleneck after optimisation is the Fusion tick overhead and emissive SetColor calls in Render(), neither of which represents a scaling concern at this player count. The scene is not GPU-bound — the orthographic camera, untextured materials, and minimal geometry mean GPU frame time is negligible. The honest limitation is that this was profiled in the Unity Editor on a development machine. A standalone build on minimum-spec hardware was not profiled, which would be a required step before any production release.

Edge case 1: NavMeshAgent vs NetworkTransform conflict:

The initial guard implementation had a NavMeshAgent driving the transform on every client alongside a NetworkTransform replicating the authority's position. Both systems write to the transform every frame. This produced jitter, teleport-back artefacts, and the guard occasionally refusing to move at all. The fix is in NetworkGuard.Spawned(). Non-authority clients disable their local NavMeshAgent entirely:

```
if (!HasStateAuthority)
{
    agent.enabled = false; return;
}
```

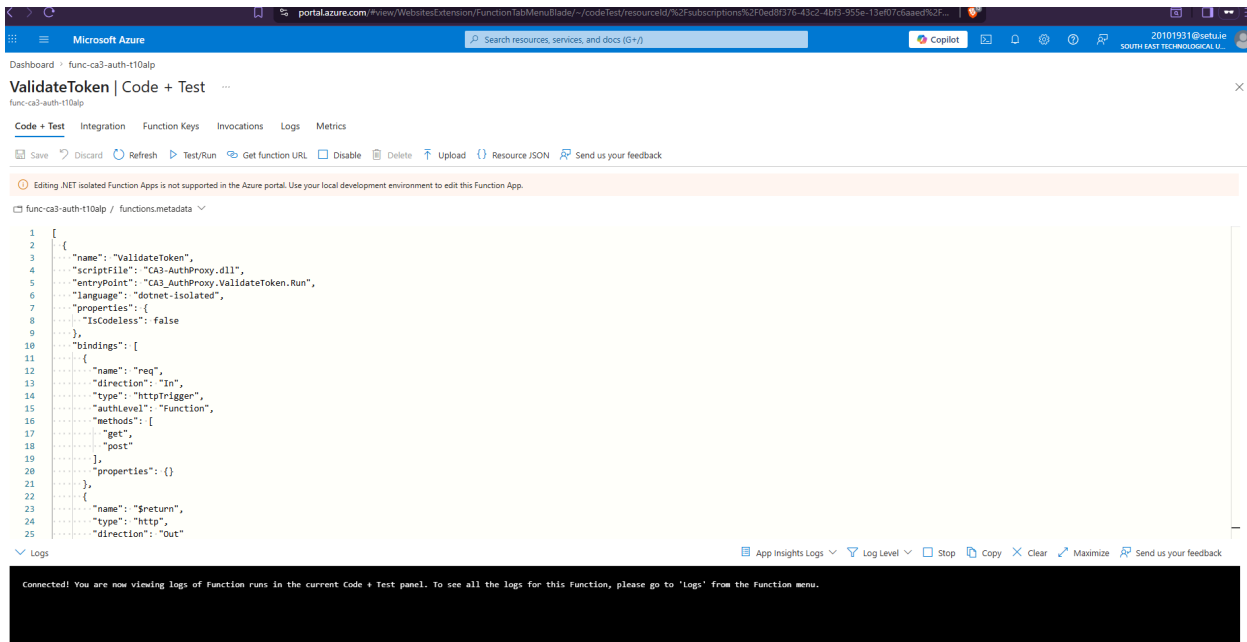
The authority client owns navigation. Remote clients receive position through NetworkTransform replication only. This is the Shared mode authority model applied at the component level and it is exactly the pattern the CA2 marker identified as misunderstood.

Edge case 2: Simultaneous pickup race in Shared mode:

Without a central host arbitrating collisions two players triggering the same pickup on the same tick would both try to write IsCollected = true and increment their own scores. I solved this using Photon's state-authority arbitration. OnTriggerEnter calls Object.RequestStateAuthority() before writing and then immediately re-checks HasStateAuthority. Photon grants authority to exactly one requester. The losing client's check fails and it early-exits without writing. This is the canonical Shared mode pattern for contested state. It is architecturally different from the CA2 HasStateAuthority guard. In Host/Client mode that guard worked because the host always won by default. In Shared mode authority must be explicitly requested and the race outcome explicitly handled.

Edge case 3: Auth/network startup race:

On first load, NetworkBootstrap.Start() ran before AuthBootstrap.Awake() had finished the anonymous sign-in. The result was [Net] AuthBootstrap missing in scene and no spawn. Polling IsSignedIn in Start() would have failed intermittently depending on async timing. I replaced the polling with an event subscription. NetworkBootstrap subscribes to AuthBootstrap.Instance.OnSignInSuccess and only calls ConnectToFusion() from that callback.



The screenshot shows the Microsoft Azure portal interface for a function app named 'func-ca3-auth-t10alp'. The 'Code + Test' tab is active, displaying the code for the 'ValidateToken' function. The code is written in C# and is a single-line function triggered by an HTTP request. The function body is enclosed in a lambda expression and calls 'CA3_AuthProxy.ValidateToken.Run'. The function is configured with a 'req' trigger and a 'return' output binding. The portal interface includes a top navigation bar with the Microsoft Azure logo and a search bar. Below the function name, there are tabs for 'Code + Test', 'Integration', 'Function Keys', 'Invocations', 'Logs', and 'Metrics'. A warning message at the top states: 'Editing .NET isolated Function Apps is not supported in the Azure portal. Use your local development environment to edit this Function App.' The code editor shows the following code:

```
1 [
2 {
3     "name": "ValidateToken",
4     "scriptFile": "CA3_AuthProxy.dll",
5     "entryPoint": "CA3_AuthProxy.ValidateToken.Run",
6     "language": "dotnet-isolated",
7     "properties": {
8         "IsCodeless": false
9     },
10    "bindings": [
11        {
12            "name": "req",
13            "direction": "In",
14            "type": "httpTrigger",
15            "authLevel": "Function",
16            "methods": [
17                "get",
18                "post"
19            ],
20            "properties": {}
21        },
22        {
23            "name": "$return",
24            "type": "http",
25            "direction": "Out"
26        }
27    ]
28 }
```

Below the code editor, there is a 'Logs' section with a message: 'Connected! You are now viewing logs of function runs in the current Code + Test panel. To see all the logs for this function, please go to 'Logs' from the Function menu.'

This handles both the already-signed-in case on scene reload and the not-yet-signed-in case on first load without any timing dependency between the two components.

Section 3: Networking Implementation

From Host/Client to Shared mode:

CA2 used `GameMode.Host` and `GameMode.Client`. The CA2 marker deducted 11 marks for this because the module required Photon Fusion 2 in Shared mode. This is not a superficial distinction. In Host/Client mode all `StateAuthority` sits with the host process. Every `HasStateAuthority` guard executes exactly once, on the host. In Shared mode authority is distributed. Each `NetworkObject` can be owned by any peer and the Fusion relay manages session state without running game logic. Writing code against Host/Client assumptions and then switching to Shared mode breaks every authority guard that relied on the host always owning a given object. CA3 corrects this throughout. `NetworkBootstrap.cs` passes `GameMode.Shared` to `runner.StartGame()` and all state-write paths are written against Shared mode authority semantics.

Pickup redesign:

CA2's `PickupItem.OnTriggerEnter` used `if (!Object.HasStateAuthority) return;`. That was correct in Host/Client because the host always held `StateAuthority` over scene-placed objects. In Shared mode, `OnTriggerEnter` is a local physics event. It fires on whichever client's simulation detects the collision, which is the player's own client, not necessarily the client holding `StateAuthority` over the pickup. A `HasStateAuthority` guard in the trigger handler would silently ignore every collision from a client that does not hold pickup authority. `NetworkPickup.cs` solves this with a two-hop RPC chain. `OnTriggerEnter` checks `score.HasInputAuthority` to confirm the colliding player belongs to this local client, then fires `RPC_RequestPickup(collector)` to `RpcTargets.StateAuthority`. The `StateAuthority` handler is the only place `IsCollected = true` is ever written. It then calls `score.RPC_AwardScore()`, also routed to `RpcTargets.StateAuthority` on the player object. Both writes are always made by the authority-holding peer regardless of session topology. The CA2 `playerScore.Score += 1` direct write in `OnTriggerEnter` was an implicit Host/Client assumption. The RPC chain removes that assumption. `IsCollected` remains a [Networked] property rather than a pure RPC event for the same reason argued in CA2. Late joiners receive the current snapshot and see already-collected pickups as absent without needing to replay any RPC history. The change from CA2 is only in how that state is written.

Codebase restructure:

CA2's `Spawner.cs` handled session setup, player spawn, despawn, and input in one class. CA3 splits this into `NetworkBootstrap.cs` (auth sequencing, session join, spawn) and `NetworkGameManager.cs` (win condition, replicated game state). Every shared CA2 concept was renamed with a `Network` prefix and given a tighter single responsibility. `PickupItem` became `NetworkPickup`, `PlayerScore` became `NetworkPlayerScore` with score awarded via `RPC_AwardScore` rather than a direct write. Six entirely new scripts were added: `AuthBootstrap`, `NetworkGuard`, `NetworkGuardVisuals`, `NetworkPlayerStun`, `NetworkPlayerAppearance`, and `NetworkGameManager`. CA2 verified that Photon Fusion 2 keeps two clients in sync over a simple pickup test. CA3 built a complete encounter on those rails.

Guard NPC:

NetworkGuard.cs wraps a NavMeshAgent with a two-state FSM (Patrol, Chase). All AI computation is gated on HasStateAuthority. The agent is disabled on non-authority clients in Spawned() so NavMesh computation happens exactly once per tick globally. Non-authority clients receive CurrentWaypointIndex and State as replicated properties and use them to drive visuals via NetworkGuardVisuals, which runs Color.Lerp in Render() on every client.



The stun mechanic uses `RPC_ApplyStun(duration)` targeted at `RpcTargets.StateAuthority`, which writes `StunUntilTime = SimulationTime + duration`. `IsStunned` is a computed property (`StunUntilTime > Runner.SimulationTime`). It is deterministic across all clients from a single replicated float with no separate networked bool needed.

Game-over synchronisation

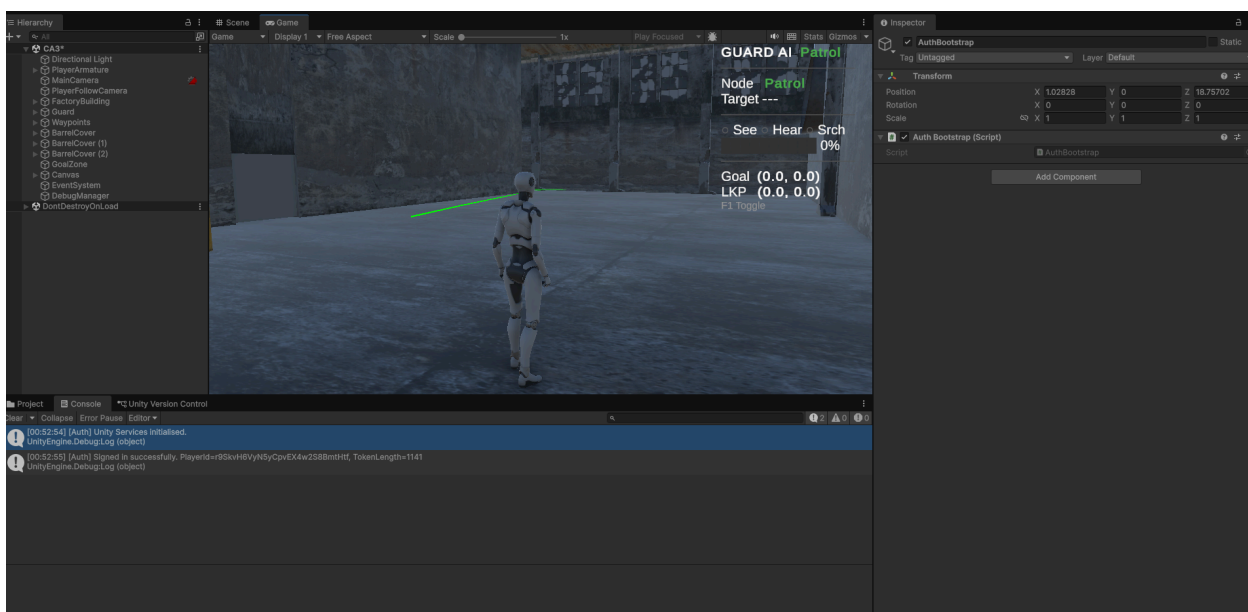
`NetworkGameManager` polls `NetworkPlayerScore.Score` in `FixedUpdateNetwork()` on the authority peer. When a player reaches `targetScore`, `WinnerPlayerId` and `GameOver` are written in the same tick. Fusion replicates both in the same state snapshot so there is no frame where `GameOver` is true while `WinnerPlayerId` is still -1. All clients' `Render()` methods activate the winner label on `GameOver == true`. This closes the CA1 catch bug directly.



The CA1 marker noted that running into the guard caused a silent freeze with no end screen. The replicated GameOver flag read in Render() means the round resolves cleanly on all clients with no Time.timeScale manipulation and no race condition

Authentication pipeline:

AuthBootstrap.cs initialises Unity Services and calls SignInAnonymouslyAsync() in Awake(). On success it fires OnSignInSuccess. NetworkBootstrap.cs subscribes to that event and only calls ConnectToFusion() after it fires. There is no code path where runner.StartGame() is invoked without a valid access token.



The token is passed to Fusion via AuthenticationValues with CustomAuthenticationType.Custom and auth.AddAuthParameter("token", AccessToken). Fusion forwards these credentials to the Azure proxy endpoint configured in the Photon App dashboard.

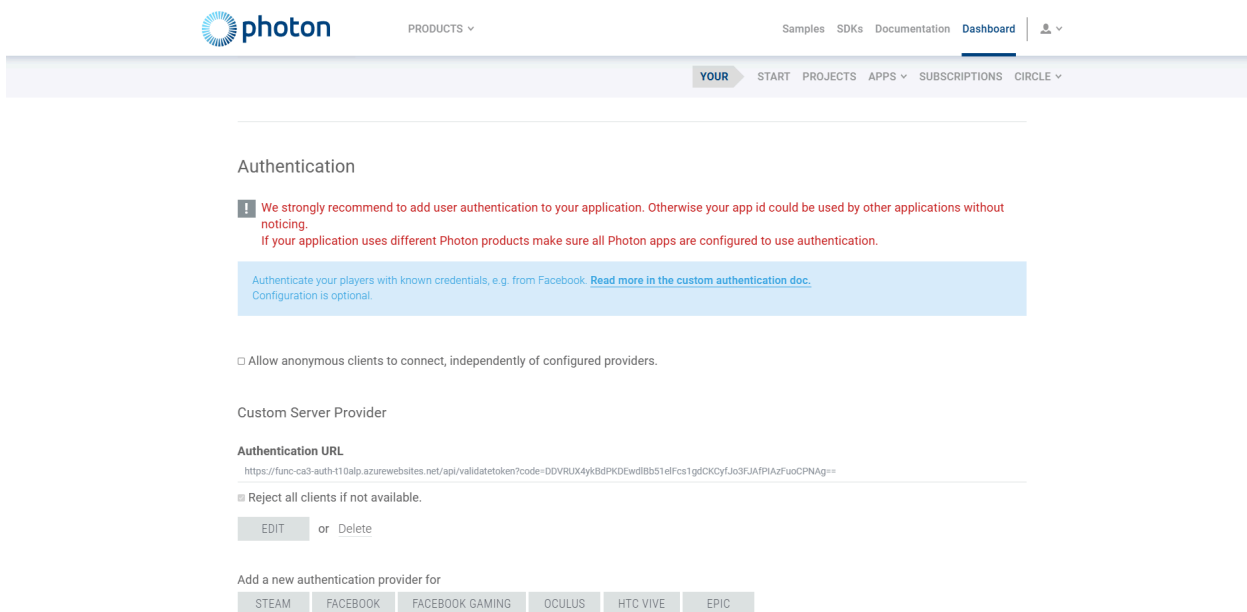
```
1  using Microsoft.Azure.Functions.Worker;
2  using Microsoft.Extensions.Logging;
3  using Microsoft.AspNetCore.Http;
4  using Microsoft.AspNetCore.Mvc;
5  using System.IdentityModel.Tokens.Jwt;
6  namespace CA3_AuthProxy;
7  public class ValidateToken
8  {
9      private readonly ILogger<ValidateToken> _logger;
10     private const string ExpectedIssuer = "https://player-auth.services.api.unity.com";
11     public ValidateToken(ILogger<ValidateToken> logger)
12     {
13         _logger = logger;
14     }
15     [Function("ValidateToken")]
16     public IActionResult Run([HttpTrigger(AuthorizationLevel.Function, "get", "post")] HttpRequest req)
17     {
18         _logger.LogInformation("ValidateToken invoked.");
19         var token = req.Query["token"].ToString();
20         if (string.IsNullOrEmpty(token))
21         {
22             _logger.LogWarning("No token in request.");
23             return BuildResponse(resultCode: 3, message: "no token provided");
24         }
25         var handler = new JwtSecurityTokenHandler();
26         if (!handler.CanReadToken(token))
27         {
28             _logger.LogWarning("Token is not a valid JWT structure.");
29             return BuildResponse(resultCode: 2, message: "malformed token");
30         }
31         JwtSecurityToken jwt;
32         try
33         {
34             jwt = handler.ReadJwtToken(token);
35         }
36         catch (Exception ex)
37         {
38             _logger.LogWarning(ex, "JWT parse failed.");
39             return BuildResponse(resultCode: 2, message: "could not parse token");
40         }
41         // Issuer check – must come from Unity Authentication.
42         if (!string.Equals(jwt.Issuer, ExpectedIssuer, StringComparison.Ordinal))
43         {
```

```

37     {
38         _logger.LogWarning(ex, "JWT parse failed.");
39         return BuildResponse(resultCode: 2, message: "could not parse token");
40     }
41     // Issuer check – must come from Unity Authentication.
42     if (!string.Equals(jwt.Issuer, ExpectedIssuer, StringComparison.Ordinal))
43     {
44         _logger.LogWarning("Issuer mismatch: {Issuer}", jwt.Issuer);
45         return BuildResponse(resultCode: 2, message: "wrong issuer");
46     }
47     // Expiry check.
48     if (jwt.ValidTo == default || jwt.ValidTo < DateTime.UtcNow)
49     {
50         _logger.LogWarning("Token expired at {Exp}.", jwt.ValidTo);
51         return BuildResponse(resultCode: 2, message: "token expired");
52     }
53     // Player id (sub claim).
54     var playerId = jwt.Subject;
55     if (string.IsNullOrEmpty(playerId))
56     {
57         _logger.LogWarning("Token has no sub claim.");
58         return BuildResponse(resultCode: 2, message: "missing player id");
59     }
60     _logger.LogInformation("Token accepted for PlayerId={PlayerId}", playerId);
61     return BuildResponse(
62         resultCode: 1,
63         message: "validated",
64         userId: playerId);
65 }
66 private static IActionResult BuildResponse(
67     int resultCode, string message, string? userId = null)
68 {
69     return new OkObjectResult(new
70     {
71         ResultCode = resultCode,
72         Message = message,
73         UserId = userId
74     });
75 }
76 }

```

The Azure Function (ValidateToken.cs) reads the token query parameter, parses it as a JWT via `JwtSecurityTokenHandler`, checks the issuer against `https://player-auth.services.api.unity.com`, and rejects the token if `jwt.ValidTo == default || jwt.ValidTo < DateTime.UtcNow`. If issuer, expiry, and the sub claim all pass, `BuildResponse` returns `{ ResultCode: 1, Message: "validated", UserId: playerId }` in an `OkObjectResult`. Any failure returns `ResultCode: 2` or `3` and Fusion closes the connection before the client joins the session.



Screenshot: Photon dashboard showing Custom Authorization endpoint configured.

Why server-side validation is necessary:

A client could pass any string as the token without ever calling `SignInAnonymouslyAsync`. If validation happened only on the client, a modified build could skip that check entirely. Putting the issuer and expiry checks inside a deployed Azure Function that the client cannot modify closes this path. The Photon App dashboard also has "Allow anonymous clients to connect" unchecked, so every connection attempt must pass the custom auth check at the Photon relay level before the session is joined.

```
PS C:\Users\Belphy\Documents\GitHub\mgd-ca1-WR\CA_Adv_3D_GameDev\CA3-AuthProxy> Invoke-RestMethod $url
resultCode message      userId
-----
3 no token provided

PS C:\Users\Belphy\Documents\GitHub\mgd-ca1-WR\CA_Adv_3D_GameDev\CA3-AuthProxy> Invoke-RestMethod "$url&token=not_a_jwt"
resultCode message      userId
-----
2 malformed token

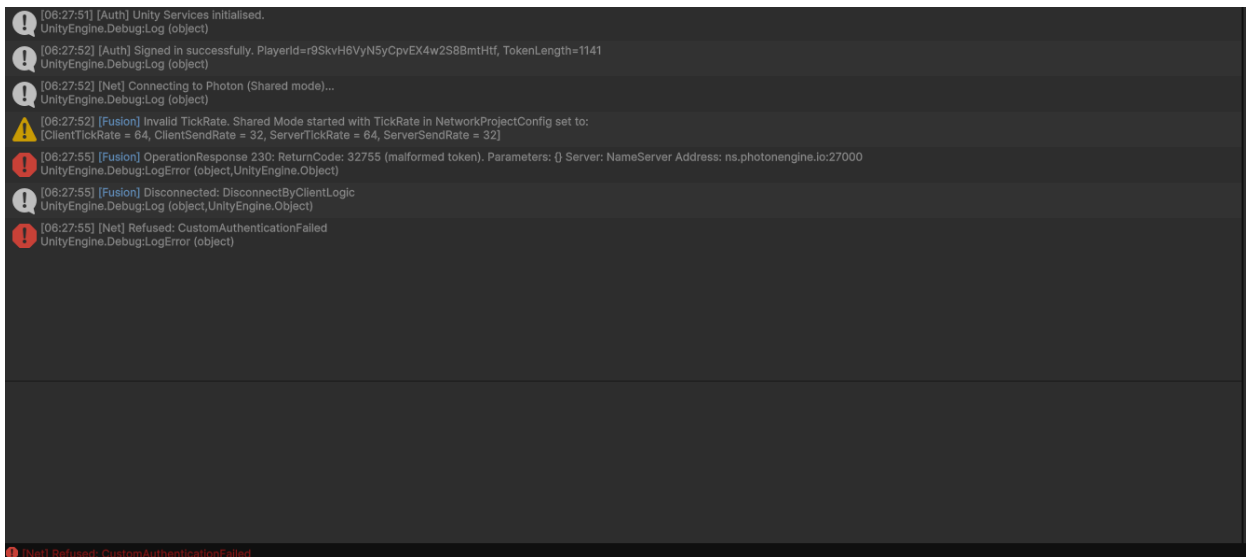
PS C:\Users\Belphy\Documents\GitHub\mgd-ca1-WR\CA_Adv_3D_GameDev\CA3-AuthProxy> Invoke-RestMethod "$url&token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI1NjM0NSIsImZlcyI6ImV4YVUwY29tIiwiaXNjaXozMDAwMDAwMDAwfQ.SfLkxwRjSMeiKf2Q74FwpMeJf36P0k6yJV_adQssw5c"
resultCode message      userId
-----
2 wrong issuer

PS C:\Users\Belphy\Documents\GitHub\mgd-ca1-WR\CA_Adv_3D_GameDev\CA3-AuthProxy> |
```

Rejection verification:

The Azure Function was tested directly via PowerShell `Invoke-RestMethod` with three deliberate failure cases: no token (ResultCode: 3, "no token provided"), a non-JWT string (ResultCode: 2, "malformed token"), and a structurally valid JWT with a wrong issuer (ResultCode: 2, "wrong issuer"). All three paths return correctly. Unity-side rejection was verified by running a client with a deliberately malformed token.

The Unity console shows Unity Services initialising and signing in successfully, then [Fusion] OperationResponse 230: ReturnCode: 32755 (malformed token), then [Fusion] Disconnected: DisconnectByClientLogic, then [Net] Refused: CustomAuthenticationFailed. The client is refused before it joins the session



Robustness and limitations:

The implementation handles the primary failure paths: lost authentication is caught before session join, NavMeshAgent conflicts on non-authority clients are resolved in Spawned(), simultaneous pickup races use RequestStateAuthority() arbitration, and the auth/network startup race is resolved via event subscription rather than polling. The most significant unhandled case is authority migration on guard disconnect. If the peer holding StateAuthority over the guard disconnects mid-session, the guard freezes for all remaining clients. Addressing this would require a HasStateAuthority vacancy check and an authority request path — either re-spawning the guard or using Fusion's Object.RequestStateAuthority() from a surviving peer. The auth failure path on the client side also has no in-game UI feedback; it logs to console only, which is sufficient for a developer environment but not for a shipped product. Both limitations are known and documented as carry-forward items.

Section 4: Networking in Modern Engines

Networking models and topologies.

Photon Fusion 2 supports three topologies: Host/Client, Dedicated Server, and Shared mode (Photon Engine, 2026). In Host/Client mode one peer process acts as the authoritative simulation. In Dedicated Server a headless process holds all authority. In Shared mode authority is distributed among peers and the relay server manages sessions without running game logic. This submission uses Shared mode, which suits small sessions of two to four players where hosting a dedicated server is not economically justified. CA2 used Host/Client and lost 11 marks for it. The distinction is not superficial. Authority model assumptions that are safe in Host/Client silently break in Shared mode. The pickup redesign in Section 3 is a direct demonstration of this. Unreal Engine 5 uses a strictly server-authoritative model. There is always a Listen Server or Dedicated Server process and clients never hold direct authority over actors (Epic Games, 2025). UE5 has no equivalent of Fusion's Shared mode because its replication model assumes a server is always present and always authoritative. Fusion's Shared mode suits small sessions without anti-cheat requirements. Unreal's strictly server-authoritative model is the right choice where a trusted authority is essential: competitive shooters, anything with economy or progression.

Lag, latency, and compensation.

The core challenge in real-time networked games is that each client must display a world that is globally consistent while responding to local input without perceived delay. Fiedler (2010) describes the tension directly: applying state only on receipt from the authority introduces input lag proportional to round-trip time. Applying state immediately from local input and correcting on receipt produces responsive local control but can cause visible corrections when the authoritative state diverges. Client-side prediction resolves this by applying local input immediately and reconciling with authoritative snapshots as they arrive

Fusion 2 implements this through its tick-based simulation model (Photon Engine, 2026). Each peer runs `FixedUpdateNetwork()` at a fixed tick rate. Fusion re-simulates forward from the last confirmed authoritative state when a correction arrives. This is why networked logic must live in `FixedUpdateNetwork()` and not in Unity's `Update()`. Code placed in `Update()` runs outside the simulation loop and cannot participate in prediction or resimulation. `Render()` is the correct location for visual interpolation that should not affect simulation state. In Unreal Engine 5, the built-in Character Movement Component provides client-side prediction and server reconciliation for character movement automatically. For arbitrary actor state, developers must implement custom prediction via Server RPCs and client-side extrapolation, which requires more work than Fusion's property-level approach (Epic Games, 2025).

Authority models compared

Fusion 2 uses `StateAuthority` (the peer permitted to write [Networked] properties) and `InputAuthority` (the peer providing simulation input) as distinct concepts on the same `NetworkObject` (Photon Engine, 2026). Unreal Engine 5 uses three roles: Server (equivalent to `StateAuthority`), Owning Client (equivalent to `InputAuthority`), and Simulated Proxy, which is all other clients observing a remotely-controlled actor (Epic Games, 2025). Fusion has no explicit Simulated Proxy role. Non-authority clients receive replicated state and render it without a

named role distinction. The CA3 pickup system makes the Fusion authority model concrete. OnTriggerEnter fires on the InputAuthority client. RPC_RequestPickup routes the state write to StateAuthority. In Unreal, the equivalent would be a Server RPC (UFUNCTION(Server, Reliable)) called from the Owning Client, with the Server performing the replicated state change. The pattern is architecturally identical. The terminology and boilerplate differ. Unreal's UPROPERTY(Replicated) with GetLifetimeReplicatedProps() and DOREPLIFETIME is more verbose than Fusion's [Networked] attribute but both achieve automatic replication from authority to all observers.

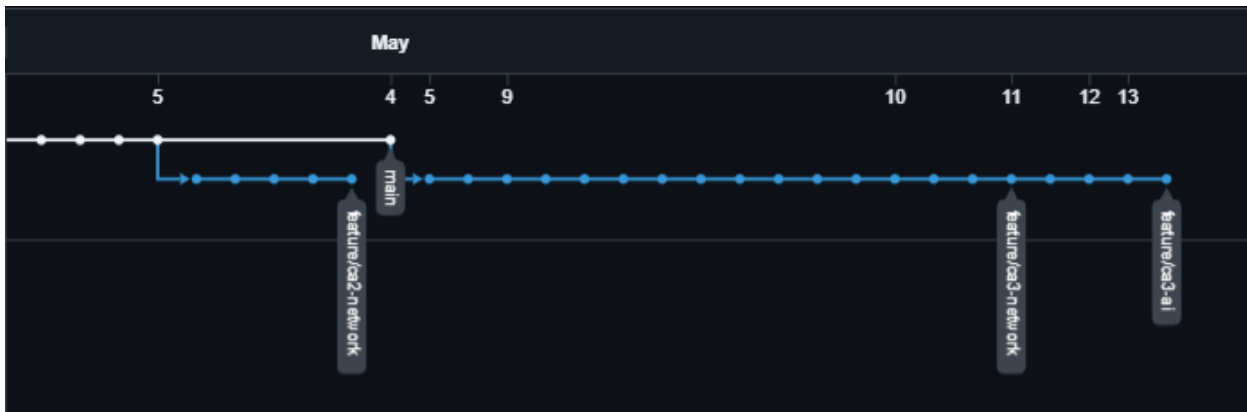
Industrial context and trends.

Hong (2024), drawing on over thirty years of multiplayer game development, identifies that the most persistent challenge in production networking is not the happy path but the long tail of edge cases: late joiners receiving a mid-game session, host or authority migration during a transaction, and the interaction between authority changes and game-state events. In CA3, IsCollected being a [Networked] property rather than an RPC event directly addresses the late-joiner case. Fusion snapshots the current state to any joining client, so a player joining mid-session sees already-collected pickups as absent without replaying any event history. Authority migration and connection drops during an active pickup RPC were not tested and represent a known limitation. The broader industry direction is toward cloud-hosted relay infrastructure, already provided by Photon, and away from direct peer-to-peer sessions driven by NAT traversal complexity and increasing regulatory pressure around game integrity (Photon Engine, 2026). State synchronisation, as implemented throughout this submission, is the dominant approach for action games because it tolerates individual client discrepancies through authority-side correction rather than requiring perfect determinism. Hong (2024) notes that junior developers entering the industry are increasingly expected to understand tick-based prediction models at an architectural level rather than treating them as black-box engine features. Fusion's explicit FixedUpdateNetwork and Render separation, and the authority concepts that govern when each fires, directly builds that understanding.

Reflection on own implementation

In a production context the single decision I would approach differently is the error handling path. The current NetworkBootstrap handles auth failure by logging an error and setting State = ConnectionState.Refused. There is no in-game UI feedback to the player. A production implementation would present a modal dialog, offer a retry, and distinguish between transient failures and permanent ones. The current implementation is sufficient for an assessed vertical slice where the developer has console access. It would not meet a production standard.

Section 5: Version Control and Professional Workflow



```
PS C:\Users\Belphe\Documents\GitHub\mgd-cal-WR\CA_Adv_3D_GameDev> git log --oneline --graph --all --decorate --no
* 8091bfb (HEAD -> feature/ca3-ai, origin/feature/ca3-ai) (ai): investigate fully visible (LKP navmesh snap, path grace window, 2s linger, look-around rotation) moved all CA3-edited scripts to CA3_AI/ and four
state screenshots
* fcd0b3f (tag: ca3-profiling-pack-ai) ai: eliminate per-frame GC alloc via cached buffers + organise profiler captures into AI/Network subfolders
* e5ab2b1 feat(ai): velocity-aware hearing + chase range gizmo + hearing ring visualisation
* 172e5eb (tag: ca3-alpha-ai) feat(ai): CA2 marker fixes + sight confidence layer, moved to CA3_AI/
* 2246676 (origin/feature/ca3-network, feature/ca3-network) tweak: network scene polish (post-gameover state, inspector tuning)
* 8ea6db6 (tag: ca3-profiling-pack) Gameover freeze all entities + profiling pack + auth refusal + azure proxy evidence
* d078333 added cinematic lighting and post-fx (aces, bloom, vignette, emissive collectibles and dynamic guard glow
* 624a165 (tag: ca3-beta) networked guard chase + stun rpc, player stun gating and 2 clients fully validated
* 2388fff networked guard with navmesh patrol, wall navigation static & baked navmesh
* c2974cf removed and ignored windows desktop.ini metadata
* bfe7983 (tag: ca3-alpha) functional working 1 client gameplay loop with raycast wall collision + screenshots
* b6bbe82 pickups, gamemanager and pipeline screenshots
* f07bf71 shared mode network scene with auth gated bootstrap, networked player movement/score/appearance
* d03a4ab Added unity authentication anonymous sign in bootstrap & integrated into CA3 scene
* f1e16f2 (tag: ca3-auth-deployed) force added auth proxy csproj over unity csproj ignore
* c07987e Scaffold azure functions auth proxy with unity token validation
* 7a152e0 Update CA3 scene with content from CA2
* de0b5f3 ignore unity auto recovery
* 6882272 imported BT system, blackboard, suspicion etc. from CA2 for AI Game Dev
* b56cf36 (tag: ca3-start) ca3 plan, repo structure, and commit convention
* d32a569 (origin/main, origin/HEAD, main) chore: restore Fusion 2 SDK, normalize line endings, set EOL policy
* 419e8c1 (origin/feature/ca2-network) Add files via upload
* 39f0829 Add new attributions for Photon Fusion and TextMeshPro
* 259a2fd Add CA2 Learning Journal with project details
* 28dada9 Add files via upload
* 5323f6f Add CA2 Test Matrix documentation
/
* ba3a7b1 (tag: ca2-submit, feature/ca2-network) Branch Strategy, Plan and UE Replication Docs aswell as Screenshots implemented
* 10ab101 created orb prefab for pickup
* dfc71f6 added basic pickup with network state and playerscore ui
* aed0951 (tag: ca2-baseline) two client baseline connect and player spawn work
* 963daa5 Spawner with session setup and player spawn and despawn
* 752520e playercharacter network prefab
* 13c977b CA2 network test scene and ground plane
* c248622 import Photon Fusion 2 SDK and App ID config
* 035725d CA2 network documentation
* f0fa163 (origin/CA2, CA2) Rename CriticalDiscussion.md to CA2_CriticalDiscussion.md
* daacfab Revise CA2 Evidence Log with session updates
* 69cdab2 Add Critical Discussion on AI Design and Lessons Learned
* d88e639 Add technical note on guard AI behavior tree and perception
* ba70614 Merge pull request #1 from Wolfgang-Romanowski/main
/
PS C:\Users\Belphe\Documents\GitHub\mgd-cal-WR\CA_Adv_3D_GameDev>
```

All 11 milestone tags are present and pushed to origin: baseline, ca1-submit, ca2-baseline, ca2-submit, ca3-start, ca3-auth-deployed, ca3-alpha, ca3-beta, ca3-profiling-pack, ca3-alpha-ai, and ca3-profiling-pack-ai. The CA3 GameDev tags were applied at meaningful points in development. ca3-start is at b56cf36, the initial scene and repo structure commit. ca3-alpha is at bfe7983, a functional single-client gameplay loop. ca3-beta is at 624a165, a stable two-client integrated session with the guard NPC. ca3-profiling-pack is at 8ea6db6, the commit that added profiler captures and the perception tick-rate optimisation. ca3-auth-deployed is at f1e16f2, precisely the commit that introduced the Azure proxy code after confirming the endpoint was live.

```

2026-05-05 ca3-start caee704 CA3 vertical slice work begins
2026-05-09 ca3-auth-deployed 879fcfa Azure Function deployed and validated against Unity issuer expectations
2026-05-09 ca3-alpha 743af04 CA3 Alpha: Single client end to end gameplay i.e. (auth + spawn + movement + pickups + win)
2026-05-10 ca3-beta cae9eaa CA3 Beta: two-client networked vertical slice with NPC patrol/chase/stun working
2026-05-10 ca3-profiling-pack b9a2384 CA3 Profiling Pack: network scene before/after + auth pipeline rejection evidence
2026-05-14 ca3-submit 538c337 CA3 Network submission

```

CA3 networking work was developed on feature/ca3-network, isolated from main and from the concurrent feature/ca3-ai branch. Active commits fell across four distinct calendar days between 5 and 11 May 2026 with the main implementation push concentrated on 9 May. Commit messages on this branch apply the planned feat:/fix:/tweak:/docs: prefix convention inconsistently. The convention is used on tuning commits (tweak: network scene polish) and the AI branch (feat(ai): velocity-aware hearing) but milestone commits use free-form descriptive subjects (networked guard chase + stun rpc, player stun gating and 2 clients fully validated). The convention was planned from the start: b56cf36 is literally titled ca3 plan, repo structure, and commit convention. It drifted under time pressure. The practical lesson is that a convention not enforced by a commit-message hook is not really a convention. I would add that hook from Week 1 on any future collaborative project. The honest assessment is that the module-wide commit distribution remains uneven. CA1 and the early CA2 period show clustering. The CA2 marker called this out directly and the CA2 learning journal acknowledged it. The CA3 period shows genuine improvement: four active days on feature/ca3-network is the most distributed commit history in the module. The discipline gained in Weeks 8 and 9 building the CA2 networking feature iteratively carried into CA3 where authentication, the guard NPC, and the pickup redesign were each committed incrementally rather than in one session.

Section 6: Overall Reflection

The most significant challenge across this module was not implementing any single system but understanding how authority model assumptions propagate silently through a networked codebase. The CA2 submission worked. Two clients connected, pickups were collected correctly, scores were synchronised. But it worked because all StateAuthority was implicitly concentrated on the host. Every HasStateAuthority guard, every direct [Networked] property write, every trigger handler assumed the host would execute it. Switching to Shared mode for CA3 meant none of those assumptions were safe. The only way to find which ones were load-bearing was to trace every state-writing operation from the point where a game event occurs to the point where a [Networked] property is assigned. The pickup system was the clearest example. The CA2 OnTriggerEnter guard if (!Object.HasStateAuthority) return; looks defensive. It reads as "only the authoritative peer processes this." In Shared mode it reads as "only the peer that happens to hold StateAuthority over this pickup will ever detect this collision." That is a silent correctness failure, not an authority guard. Recognising this required understanding that OnTriggerEnter is a local physics event, not a networked callback. The RPC chain in NetworkPickup.cs is the pattern that emerges from that understanding. Writing it was the moment the Shared mode authority model became concrete rather than abstract. The decision I am most satisfied with is the guard's emissive state feedback. Having the guard's _EmissionColor interpolate from near-black to HDR red as it enters chase, driven by a [Networked] enum that replicates to all clients, meant game state, visual feedback, and network synchronisation were all handled by the same data path. A player on any client sees the guard glow red when it detects them. No separate alert system. No UI indicator. No additional networked property.

The CA1 rendering work served a functional gameplay role in CA3 rather than being decorative. That is the right outcome for an integrated vertical slice. One outcome worth naming is the closure of a bug from the CA1 feedback. The CA1 marker noted that running into the guard caused a silent freeze with no end screen. NetworkGameManager addresses this directly. [Networked] GameOver and WinnerPlayerId are written in a single tick by the authority peer and read in Render() on all clients to display the winner label. No Time.timeScale manipulation, no race condition on the end state, round resolves cleanly regardless of which client triggers it. If I were starting again with the knowledge I have now I would make two changes. The first is mundane: test with two clients from the first hour of networking development, not after the feature is done. In both CA2 and CA3 the most significant bugs only surfaced under two-client conditions. Single-client testing creates false confidence. The second is architectural: I would design the guard to use Fusion's authority request pattern so StateAuthority could migrate cleanly if the peer holding it disconnected mid-session. The current guard freezes if that peer drops. Addressing it requires a HasStateAuthority vacancy check and a re-spawn or authority-request path. That is the most concrete production gap in the current implementation. The module's progression from CA1 to CA2 to CA3 maps onto a trajectory from rendering craft to systems thinking to integration judgement. CA1 was about disciplined application of known techniques. CA2 introduced the complexity of distributed state, where correctness depends not on what the code does in isolation but on which peer executes it and when. CA3 required holding all of that simultaneously: rendering, navigation, BT decision-making from the AI module, and the networking layer in a single scene with each system interacting with the others. The guard's chase state is written by AI logic, replicated via Fusion, read by a visual component, and communicated to the player through emissive colour driven through ACES

post-processing. That chain running correctly is the best evidence I can offer that the integration was genuine rather than assembled from disconnected components.

Attribution and AI Use Statement

Third-party packages and assets: Photon Fusion 2 SDK (v2.0.12). Photon Engine GmbH. Free tier. <https://www.photonengine.com/fusion>. TextMeshPro (com.unity.textmeshpro). Unity Technologies. Included via Package Manager. Unity Authentication SDK (com.unity.services.authentication). Unity Technologies. Included via Package Manager. Module-provided PBR texture set from the SETU module repository (Common Concrete Wall 01 and Windows Broken 01 series). Used as supplied with no modifications to texture data. AI assistance: Claude (Anthropic) was used during development for debugging assistance, assisted implementation of the networked guard NPC, and research support particularly around Azure Functions setup and the Photon Custom Authorization pipeline. All implementation decisions, written work, and submitted code are my own. The AI was used as a sounding board and research aid rather than a code generator.

References

Epic Games (2025) Networking Overview for Unreal Engine.

Available at:

<https://dev.epicgames.com/documentation/en-us/unreal-engine/networking-overview-for-unreal-engine>

[Accessed 13 May 2026].

Fiedler, G. (2010) What Every Programmer Needs To Know About Game Networking. Gaffer On Games.

Available at:

https://gafferongames.com/post/what_every_programmer_needs_to_know_about_game_networking/

[Accessed 13 May 2026].

Hong, C. (2024) Tips to Overcoming Network Latency in Real-Time Multiplayer. devcom Developer Conference 2024. Available at:

<https://www.youtube.com/watch?v=dLEwFfWats8>

[Accessed 13 May 2026].

Photon Engine (2026) Fusion 2 Documentation: Network Topologies.

Available at:

<https://doc.photonengine.com/fusion/current/manual/network-topologies>

[Accessed 13 May 2026]